

Commuting Taylor projections: the general- d algebraic skeleton

Claude, with Daniel Murfet

2026-09-07

Abstract

For a sequence of linear "Taylor projection" operators P_n on a module, the recursion $R_0 = I$, $R_{n+1} = (I - P_n)R_n$, $F_0 = 0$, $F_{n+1} = F_n + P_n R_n$ gives the exact decomposition $I = F_n + R_n$ with no commutation or idempotence hypotheses (Astra #6(d)). For $n = 2$ this is the rectangular face-jet decomposition $F_2 = P_0 + P_1 - P_1 P_0$, $R_2 = I - P_0 - P_1 + P_1 P_0$ of unit 106. The analytic instantiation is coefficient truncation on multi-index arrays: truncProj_ℓ^M keeps the coefficients with $\alpha_\ell < M$; these are linear, idempotent and commute, and the remainder of the first d truncations is the tail indicator $\alpha_\ell \geq M_\ell$ for all $\ell < d$. This is the general- d skeleton on which the product-density and finite-part calculus will be hung. Zero `sorry/axiom`.

1 The things formalised

```
noncomputable def remOp (P : → E → [] E) : → E → [] E
| 0 => LinearMap.id
| n + 1 => (LinearMap.id - P n) remOp P n
noncomputable def faceOp (P : → E → [] E) : → E → [] E
| 0 => 0
| n + 1 => faceOp P n + P n remOp P n
theorem faceOp_add_remOp (P : → E → [] E) (n : ) :
  faceOp P n + remOp P n = LinearMap.id
theorem remOp_two (P) : remOp P 2 = LinearMap.id - P 0 - P 1 + P 1 P 0
theorem faceOp_two (P) : faceOp P 2 = P 0 + P 1 - P 1 P 0
noncomputable def truncProj (M : ) : (( → ) → ) → [] (( → ) → )
-- c fun => if < M then c else 0
theorem truncProj_idem / truncProj_comm
theorem remOp_truncProj_apply (M : → ) (d : ) (c : ( → ) → ) ( : → ) :
  remOp (fun => truncProj (M)) d c = if < d, M then c else 0
```

2 Mathematics

$F_{n+1} + R_{n+1} = F_n + P_n R_n + R_n - P_n R_n = F_n + R_n$, so the decomposition holds by induction using only additivity. Unfolding two steps gives the rectangular operators, with the corner term $P_1 P_0$ (the v -projection applied to the u -face jets) exactly as in unit 106, where c_{ij} were defined as the v -jets of the u -faces. For truncations, $(I - P_d)x(\alpha) = x(\alpha)$ if $\alpha_d \geq M_d$ and 0 otherwise, so R_d restricts the array to the orthant $\alpha_\ell \geq M_\ell$ ($\ell < d$).

3 Paper-facing meaning

This is the operator form of the inclusion–exclusion over faces of the box in `thm:TaylorTree`: the block integral of an amplitude splits as $\sum_S(\pm)$ face contributions (polynomial in the coordinates

of S) plus a remainder vanishing to order M_ℓ in every coordinate, and for a coefficient array the remainder is literally the tail of the double (multiple) series. The $d = 2$ theory (units 104–115) is the case $d = 2$; the general- d analytic content still to come is the product density $r^{p-1} \log(R/r)^{d-1} / ((d-1)! \prod k_\ell)$ and the iterated finite parts.

4 Lean notes

- `abel` closes the operator identities in the additive group of linear maps after `LinearMap.sub_comp, comp_sub, id_comp, comp_id`.
- `simp` normalises `< d + 1` to `d`, defeating hypotheses stated in the former form; rewrite the ifs explicitly with `if_pos/if_neg` and keep the conditions verbatim.
- A `LinearMap` defined by `where toFun ... map_add' ... map_smul'` with pointwise `ite` needs `funext` and `split_ifs <;> simp` in both fields.